

Проект Accessible First

Идея

Accessible First — это экосистема инструментов для разработки современных веб-сайтов и веб-приложений, в которой доступность, качество, производительность и простота использования являются не дополнительными возможностями, а фундаментальными принципами.

Цель проекта — сделать создание качественных интерфейсов максимально простым, а создание плохих интерфейсов — сложным или невозможным.

Проект не ставит задачу заменить React, Vue или другие технологии. Главная задача — предоставить разработчикам систему, позволяющую создавать надежные, быстрые и доступные приложения без необходимости постоянно помнить о сотнях требований WCAG, ARIA и особенностях работы скринридеров.

Философия проекта

Accessible First

Доступность является стандартом, а не дополнительной функцией.

Все компоненты, механизмы и инструменты проектируются таким образом, чтобы корректная работа со скринридерами, клавиатурой и вспомогательными технологиями была встроена по умолчанию.

Простота

API должен быть понятен начинающему разработчику.

Хорошее решение должно быть проще плохого.

Количество настроек должно быть минимальным, а значения по умолчанию должны быть оптимальными.

Надежность

Компоненты должны работать предсказуемо.

Обновления не должны ломать существующий код.

Предпочтение отдается стабильности, а не модным тенденциям.

Производительность

Минимальные зависимости.

Минимальный размер сборки.

Быстрый рендеринг.

Поддержка статических сайтов и приложений.

Независимость

Архитектура проекта должна позволять использовать различные технологии.

Ни одна внешняя библиотека не должна быть критически важной частью системы.

В будущем реализация может быть заменена без изменения публичного API.

Архитектура

Проект состоит из нескольких уровней.

Core

Независимое ядро.

Содержит:

- управление фокусом;
- навигацию с клавиатуры;
- live regions;
- управление модальными окнами;
- генерацию идентификаторов;
- проверки доступности;
- механизмы событий.

Core не зависит от React, Vue или других фреймворков.

Components

Набор готовых компонентов.

Например:

Button

Input

Checkbox

RadioGroup

Dialog

Tabs

Menu

Accordion

Table

Breadcrumb

Select

Tooltip

Toast

TreeView

DatePicker

Каждый компонент:

- соответствует WCAG;
- поддерживает клавиатурную навигацию;
- корректно работает со скринридерами;
- поддерживает масштабирование;
- имеет разумные значения по умолчанию.

Layout System

Система построения интерфейсов.

Header

Sidebar

Container

Stack

Grid

Card

Section

Main

Footer

Layout должен обеспечивать:

- адаптивность;
 - правильную семантику;
 - логичную структуру документа;
 - удобную навигацию.
-

Forms

Подсистема форм.

Автоматически обеспечивает:

- связь label и input;
 - сообщения об ошибках;
 - aria-describedby;
 - required;
 - валидацию;
 - работу со скринридерами.
-

Accessibility Tools

Инструменты контроля качества.

Во время разработки система должна предупреждать:

- отсутствие названия у кнопки;
 - отсутствие alt;
 - неправильную структуру заголовков;
 - проблемы с фокусом;
 - недостаточный контраст;
 - отсутствие описания у элементов формы;
 - нарушения WCAG.
-

Themes

Система тем оформления.

Поддержка:

- светлой темы;
 - темной темы;
 - высокого контраста;
 - увеличенных размеров;
 - пользовательских настроек.
-

Documentation

Документация является частью проекта.

Для каждого компонента должны присутствовать:

- описание;
 - примеры;
 - поддерживаемые клавиши;
 - работа со скринридерами;
 - соответствующие критерии WCAG;
 - типичные ошибки.
-

Долгосрочная стратегия

Этап 1

Создание независимого Core.

Без привязки к React.

Этап 2

Создание библиотеки компонентов.

Возможно, сначала на React для ускорения разработки.

Этап 3

Разработка адаптеров.

React Adapter

Vue Adapter

Web Components Adapter

Svelte Adapter

Публичный API должен оставаться одинаковым.

Этап 4

Создание собственной дизайн-системы.

Этап 5

Создание инструментов автоматической проверки доступности.

Этап 6

Создание собственной системы шаблонов и генерации приложений.

Этап 7

При необходимости создание собственного фреймворка.

Но только в том случае, если существующие технологии начнут ограничивать развитие проекта.

Главный принцип

Разработчик не должен помнить сотни требований WCAG.

Он должен пользоваться простыми компонентами, а система должна брать сложность на себя.

Хороший интерфейс должен получаться естественным образом.

Миссия

Сделать создание качественных, быстрых и доступных веб-приложений простым и естественным.

Создать экосистему, в которой доступность является нормой, а не дополнительной задачей.

Не бороться с существующими технологиями, а использовать лучшие решения, постепенно создавая независимую, надежную и долгоживущую платформу.